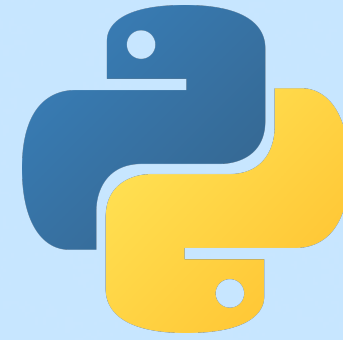


Introduction and basics

Command line, Python, Jupyter

Amine Ferdjaoui
amine.ferdjaoui@sogeti.com

Contenu du cours

1. Manipulation de la ligne de commande via terminal.

- a. Chemins
- b. Commandes Bash
- c. Editeur de text VIM

2. Python

- 1. Pip et venv

[New] 2. Uv

[New] 3. Outils de Coding avec IA dans VS Code

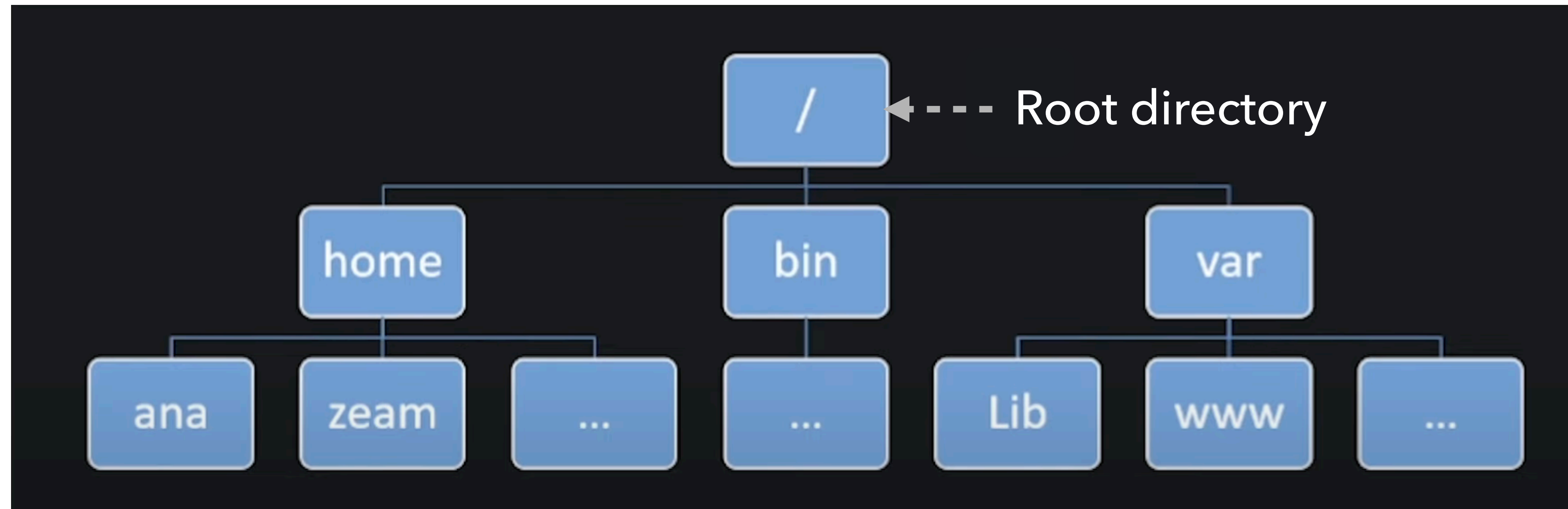
3. Jupyter et *voilà*

A vertical blue gradient bar on the left side of the slide, transitioning from a darker blue at the top to a lighter blue at the bottom.

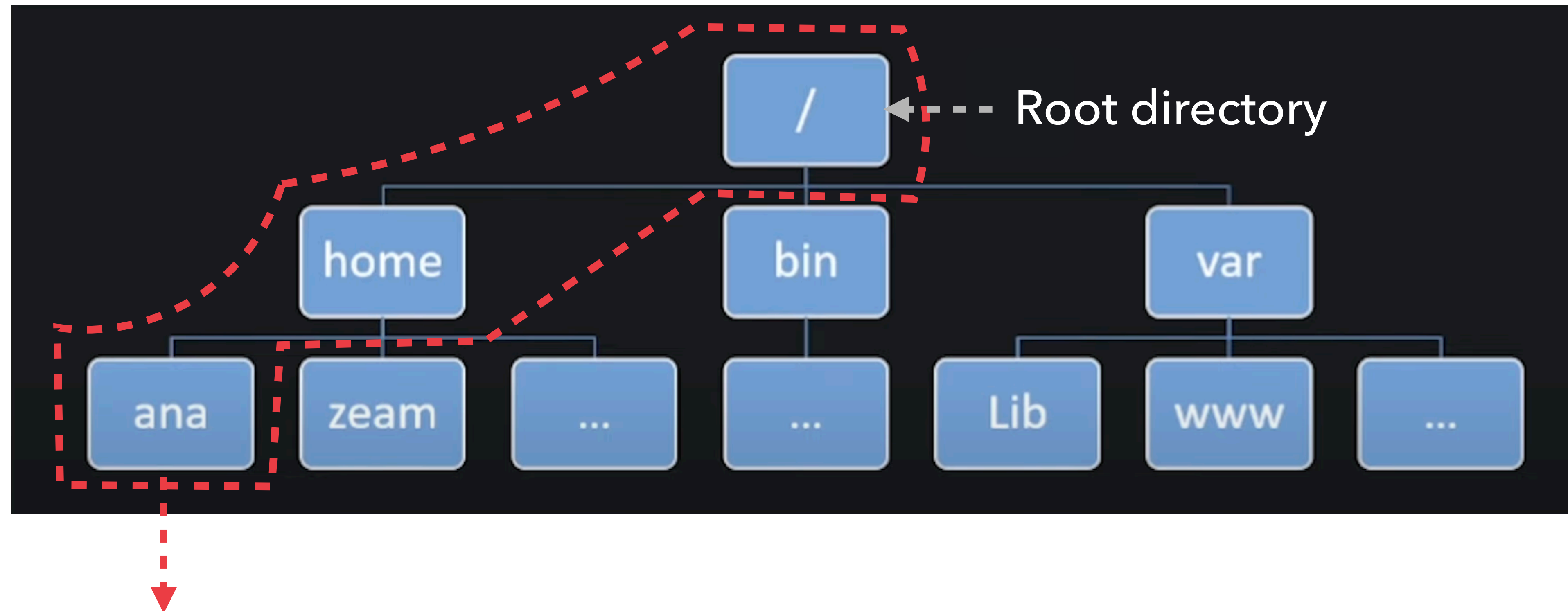
Linux

Command line

Linux file system

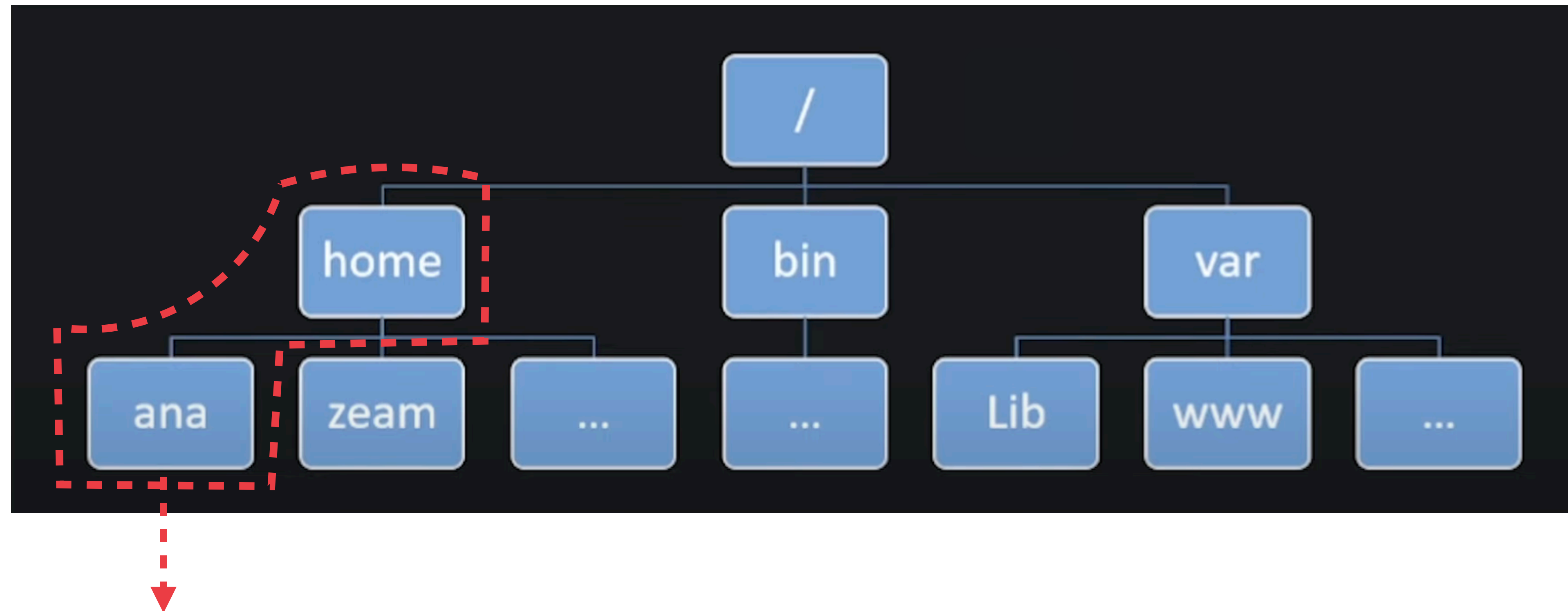


Linux file system



Absolute path **/home/ana** (Full path from root)

Linux file system



Relative path (Depends on your current directory)

Command line basics

```
$ pwd    #Print working directory
```

```
$ cd <dir> #Change directory
```

```
$ cd ..  #Change to parent directory
```

```
$ ls -A  #List
```

```
$ mkdir <dir(s)> #Create one or multiple directories
```

```
$ touch <file(s)> #Create one or multiple files
```

Command line basics

\$ pwd #Print working directory

\$ cd <dir> #Change directory

\$ cd .. #Change to parent directory

\$ ls -A #List

\$ mkdir <dir(s)> #Create one or multiple directories

\$ touch <file(s)> #Create one or multiple files

\$ cp <src> <dest> #Copy file

\$ cp -r <src> <dest> #Copy directory

\$ mv <src> <dest> #Move/rename file

\$ rm <file> #Remove file

\$ rm *.txt #Remove all text files

\$ rm -r <dir> #Remove directory

Command line basics

```
$ history #history > hist.txt
```

```
$ clear
```

```
$ echo
```

```
$ chmod #Change file permissions
```

```
$ grep #history | grep cd
```

```
$ diff #Compare two text files line by line
```

Command line basics

```
$ man <command> #Manual page
$ locate <file> #Locate a file in the system
$ cat <file> #Display a file in the terminal
$ open . #(or start .) display the current folder
$ head #Show the first lines of a file
$ tail #Show the last lines of a file
$ exit #Exit the current shell/terminal session
$ ps # (Process Status) Show the list of running processes
$ kill #Terminate a process by its Process ID (PID)
```

```
$ history #history > hist.txt
$ clear
$ echo
$ chmod #Change file permissions
$ grep #history | grep cd
$ diff #Compare two text files line by line
```

1. Create your workspace

- Go to your **home directory** (`pwd` to check).
- Create a folder named `M1_project` (`mkdir`).
- Inside it, create 3 subfolders:
 - `data/` → for datasets
 - `src/` → for Python scripts
 - `notebooks/` → for Jupyter notebooks

2. Add some files

- Inside `src/` create two files: `main.py` and `utils.py` (`touch`).
- In `data/` create two files: `train.csv` and `test.csv`.
- Verify that everything is created (`ls -A`).

3. Organize the files

- Move `main.py` into the root folder `M1_project/` (`mv`).
- Copy `utils.py` into `notebooks/` (`cp`).
- Rename `train.csv` into `training.csv`.

4. Clean up mistakes

- You don't need `test.csv`. Delete it (`rm`).
- Also, remove the `utils.py` inside `notebooks/` using `rm`.

5. Explore history and filters

- Show the list of your previous commands (`history`).
- Save them into a file called `history.txt`.
- From this file, extract only the commands where you used `cd` (`grep`).

Use VIM

VI (**V**isual editor) and VIM (**V**i **I**Mproved) are default text editor on almost all Unix/Linux systems.

```
$ vim main.py
```

The modes are shown at the bottom left of the editor

- The **command mode** is the default mode.
- To go back to command mode type **esc** key.
- To start modifying (**Insert mode**) type **i** key on keyboard.
- To **exit**, go back to command mode and
 - no modifications, type **:q**
 - modified file
 - save modifications **:wq**
 - exit without save **:q!**

Python

Virtual environment

When you start a project locally you should always work with **virtual environments (venv)** or conda.

To initialize an empty virtual environment :

```
$ python -m venv venv
```

To start it :

Unix (mac or linux)

```
$ source venv/bin/activate
```

On windows

```
$ .\venv\Scripts\activate
```

To exit :

```
(venv) $ deactivate
```

How to use python?

- Command line REPL (Read-Eval-Print Loop)

\$ python

```
Python 3.12.13 (main, Mar 25 2026, 03:16:06) [Clang 22.1.1 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

\$ ipython

```
Python 3.12.13 (main, Mar 25 2026, 03:16:06) [Clang 22.1.1 ]
Type 'copyright', 'credits' or 'license' for more information
IPython 9.17.1 -- An enhanced Interactive Python. Type '?' for help.
Tip: You can change the editing mode of IPython to behave more like vi, or emacs.

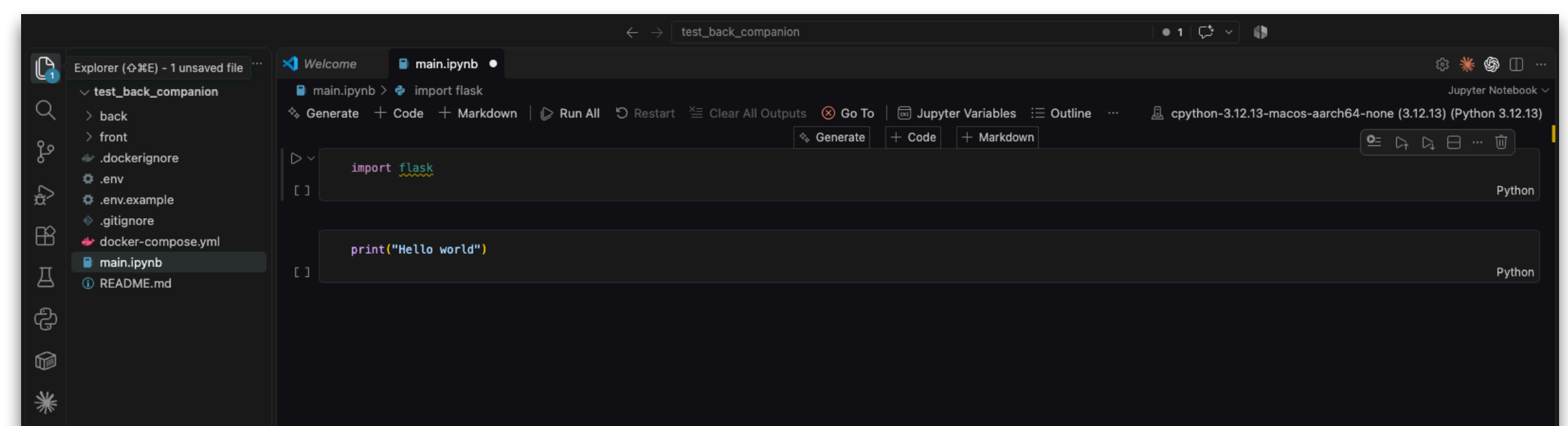
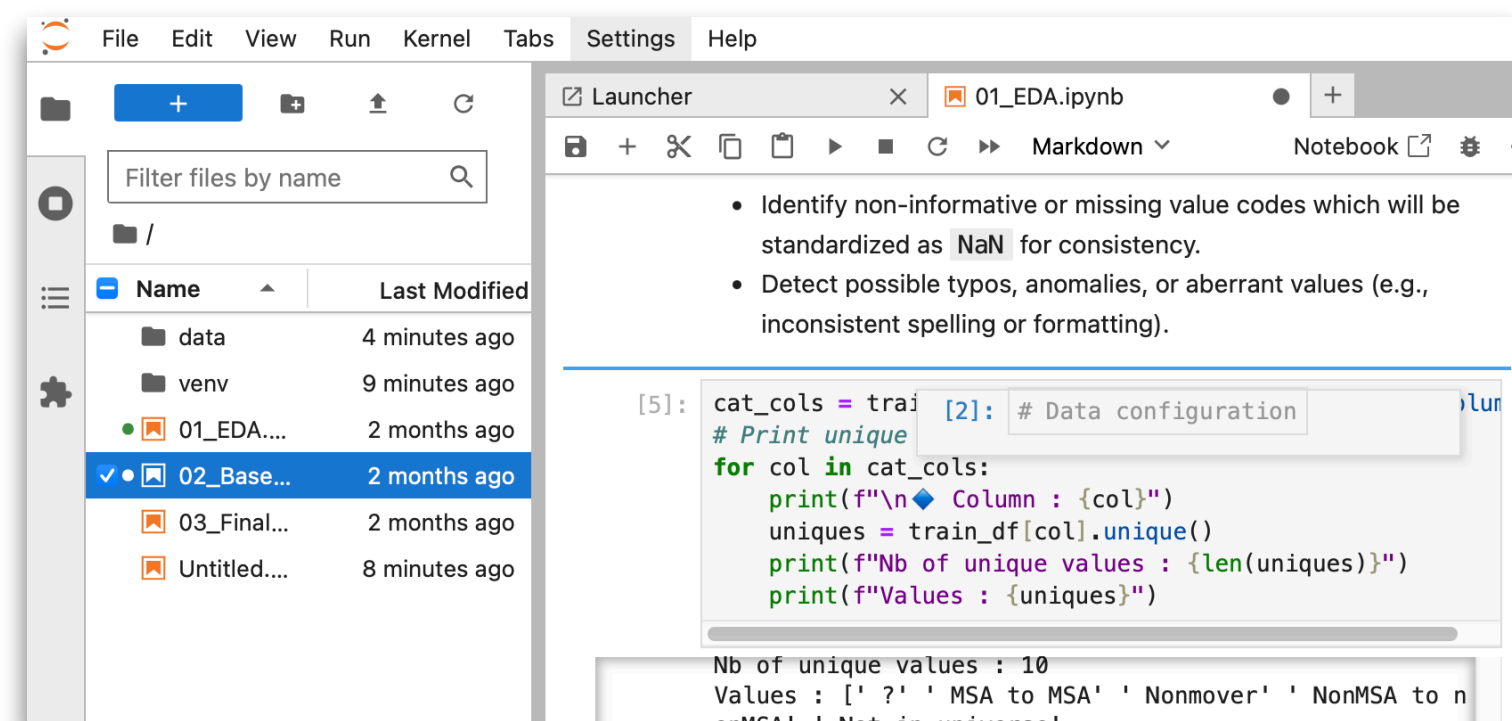
In [1]: █
```

- Python scripts (.py files)

\$ python main.py

- Jupyter Notebook / .ipynb files

IDEs, like PyCharm, VS Code.



PIP : Package Installer for Python

Install a library

```
$ pip install <library_name>
```

```
$ pip install pandas
```

```
$ pip install pandas==2.2.2
```

Install a library list from file

```
$ pip install -r requirements.txt
```

requirements.txt

```
1 matplotlib==3.10.6
2 numpy==1.26.4
3 pandas==2.2.2
4 Flask==3.1.2
```

Show installed packages

```
$ pip freeze
```

```
$ pip list
```

Remove a library

```
$ pip uninstall <library_name>
```

```
$ pip uninstall pandas
```

Context : You need to write a **Python script** for the project. But you must do it properly:

- edit it using Vim,
- run it in a **virtual environment**,
- manage dependencies with `pip`,
- and control file permissions with `chmod`.

1. Write your first script with Vim

- Go inside the `M1_project/` folder created in TP1.
- Create a new file `hello.py`. 
- Open it with Vim and write a simple program

2. Make the script executable

- Check permissions using `ls -l`
- Add executable permission using `chmod +x`
- Remove the executable permission and try to run again.
- Re-add the executable permission.

3. Create a virtual environment and install libraries

- Initialize a venv inside the project.
- Inside the venv, install pandas.
- Freeze the environment into a requirements file.
- Deactivate the venv and Create a new venv and reinstall from requirements.

```
import sys
import pandas as pd

def greet(name):
    print(f"Hello {name}, welcome to M1 project!")

def show_grades():
    df = pd.read_csv("data/students.csv")
    print("Students and grades:")
    print(df)

if __name__ == "__main__":
    if len(sys.argv) > 1:
        if sys.argv[1] == "grades":
            show_grades()
        else:
            greet(sys.argv[1])
    else:
        print("Hello World")
```

TP2 : First Python Script with Virtual Environment

Context : You need to write a **Python script** for the project. But you must do it properly:

- edit it using Vim,
- run it in a **virtual environment**,
- manage dependencies with `pip`,
- and control file permissions with `chmod`.

1. Write your first script with Vim

- Go inside the `M1_project/` folder created in TP1.
- Create a new file `hello.py`.
- Open it with Vim and write a simple program

2. Make the script executable

- Check permissions using `ls -l`
- Add executable permission using `chmod +x`
- Remove the executable permission and try to run again.
- Re-add the executable permission.

```
ls -l hello.py
chmod +x hello.py
./hello.py Amine
chmod -x hello.py
```

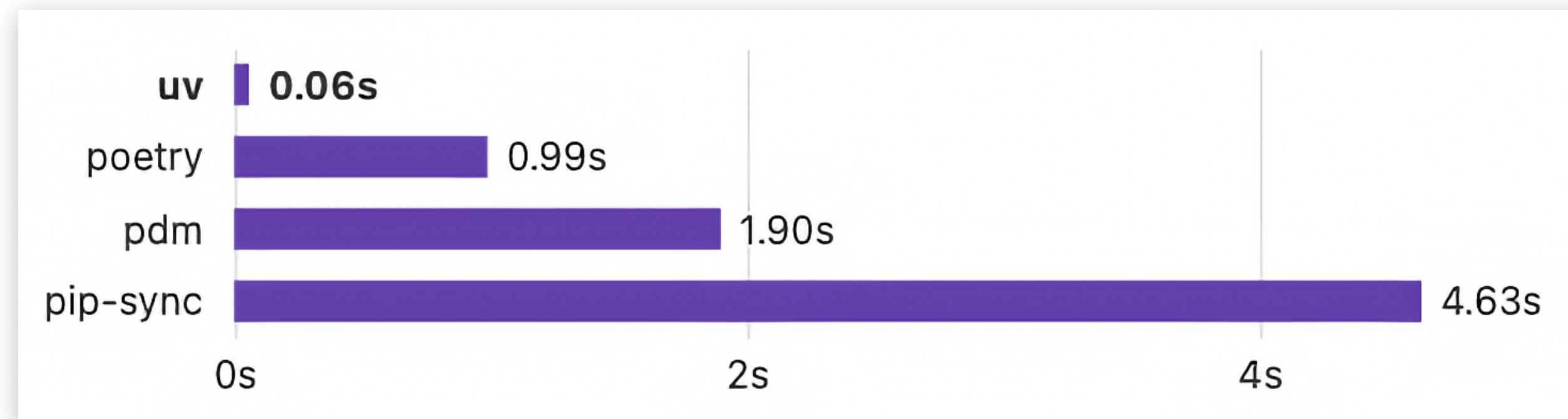
3. Create a virtual environment and install libraries

- Initialize a venv inside the project.
- Inside the venv, install pandas.
- Freeze the environment into a requirements file.
- Deactivate the venv and Create a new venv and reinstall from requirements.

```
pip install numpy pandas
pip freeze > requirements.txt
python -m venv venv2
source venv2/bin/activate
pip install -r requirements.txt
```

Python - uv

- An extremely **fast** Python package and project manager.
- It automates and makes the environment, dependencies, execution cycle **easier** and more **reliable**.



Classic workflow

- ◆ create a venv
- ◆ activate it
- ◆ install with pip
- ◆ export requirements.txt
- ◆ hope to reproduce the same environment

Uv

- ◆ creates/manages .venv
- ◆ adds dependencies
- ◆ runs code in the right environment
- ◆ locks exact versions
- ◆ speeds up installs with cache

Pyproject.toml: declared dependencies

uv.lock exact resolved versions

.venv/ local project environment

Python - uv essential commands

Set python version:

```
$ uv python install 3.12  
$ uv python pin 3.12 #tells the project which Python version to use
```

Create / initialize a project:

```
$ uv init my_project #for an existing project use only uv init
```

To add or remove a dependency only use:

```
$ uv add flask  
$ uv remove flask
```

→ Dependencies are recorded in
`pyproject.toml` and resolved in
`uv.lock`.

To run python script only use:

```
$ uv run main.py
```


AI coding tools

1) Classic prompting workflow

AI interface → copy/paste → local execution



Often missing: environment setup • dependencies • tests • README • Git hygiene

Good for exploring an idea, but the developer must rebuild the project context manually.

2) In the IDE / VS Code

Agent works inside the project: files, terminal, tests, Git

VS Code / local project

```
src/app.py
pyproject.toml
uv.lock
tests/test_app.py
```

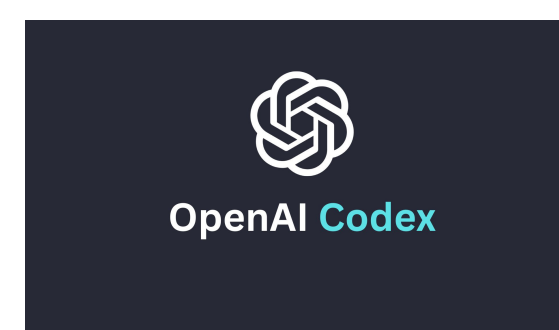
Agent actions

- edits files
- runs commands
- explains errors
- proposes fixes

Stronger workflow — but still requires supervision:

check requirements, review diffs, run tests, control versions.

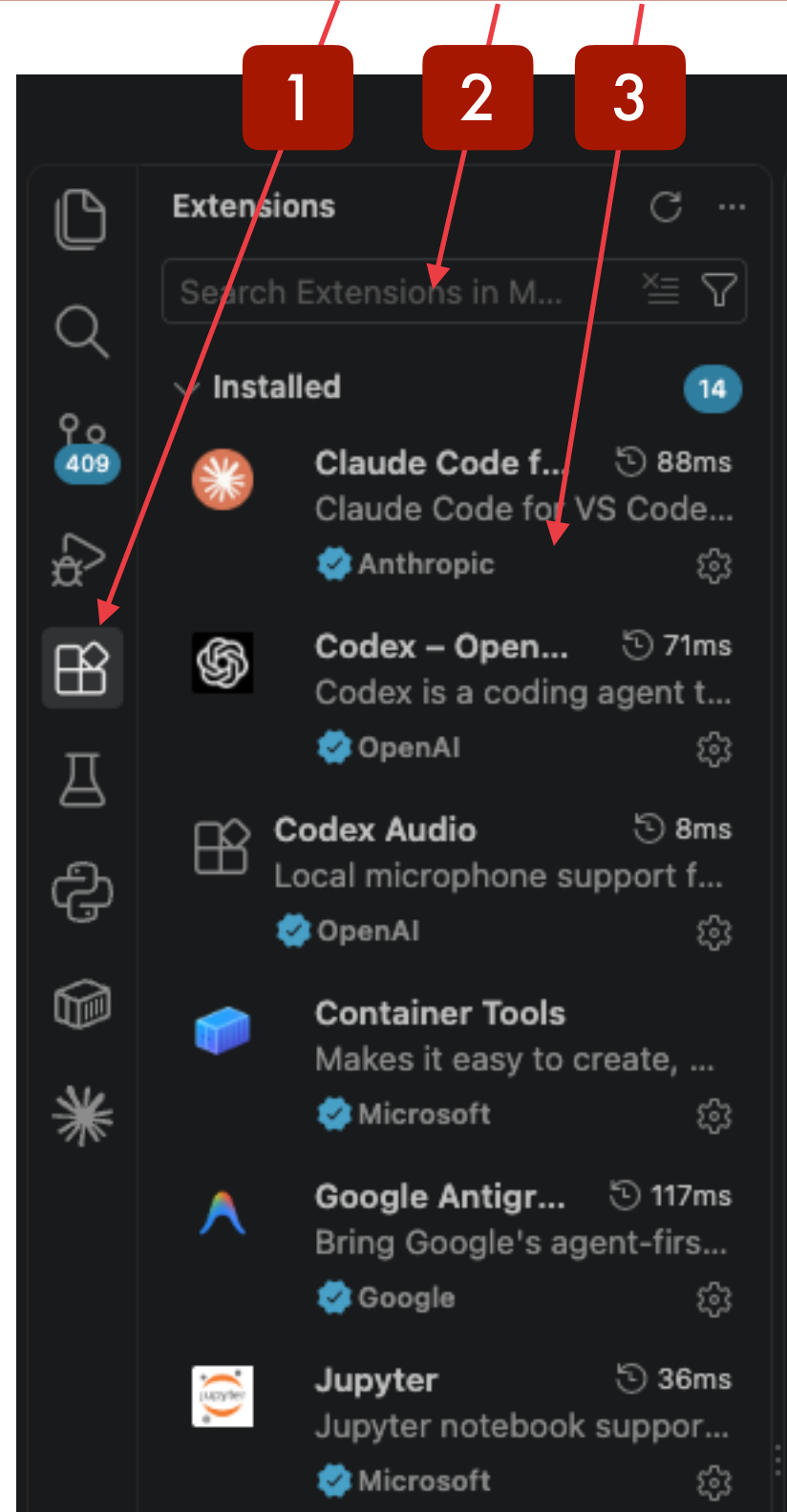
Representative coding agents / providers



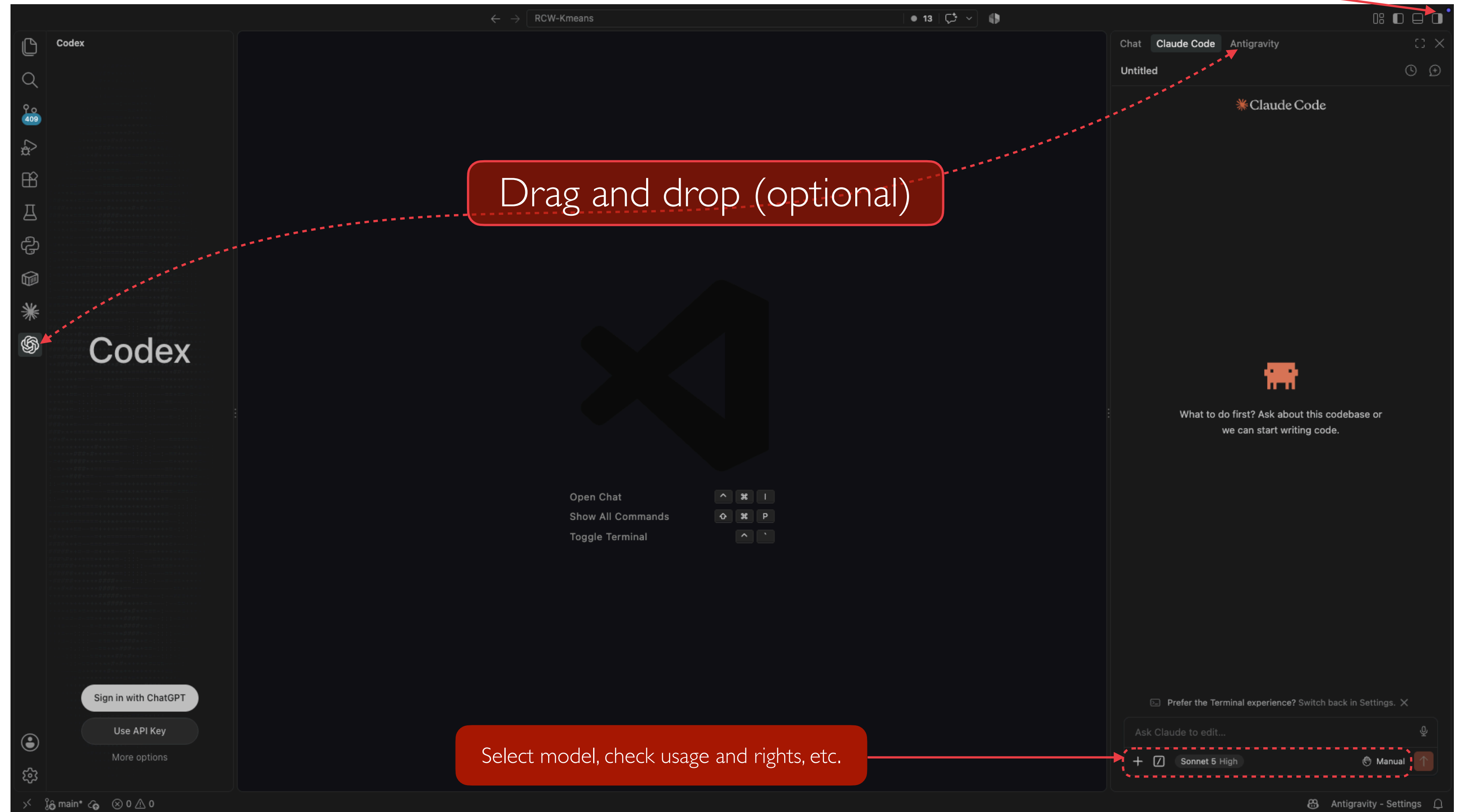
AI can generate and modify code, but you remain responsible for the project: environment, dependencies, reproducibility and version control.

AI coding tools

Add extension here



Open the AI panel



Jupyter

Jupyter

- Open-source project (from IPython, 2014)
- Stands for **JU**lia, **PY**Thon, **R** (multi-language support)
- Ecosystem for interactive computing and reproducible research
- Includes Notebook, Lab, kernels, and extensions

Jupyter Notebook

- Classic web interface for creating .ipynb notebooks
- Combines code, text (Markdown), and visualizations
- Limitations: single-window interface, less flexible for large projects

JupyterLab

- Modern interface, successor of Jupyter Notebook
- Multi-tab and multi-panel workspace (notebooks, terminals, text and csv files)
- More powerful for complex data science and ML projects
- Heavier than classic Notebook

Installation

```
$ pip install jupyterlab
```

Start jupyter lab

```
$ jupyter lab #or jupyter notebook to use only jupyter notebook
```

By default jupyter starts in port localhost:8888

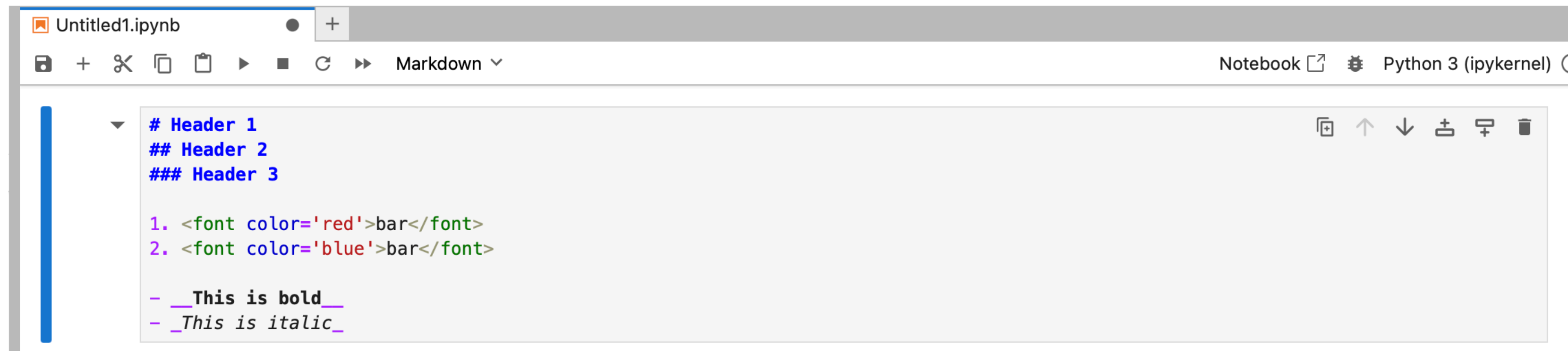
To change default port

```
$ jupyter lab --port=9999
```

To change access token

```
$ jupyter lab --NotebookApp.token='0123456789'
```

Jupyter Lab : Markdown

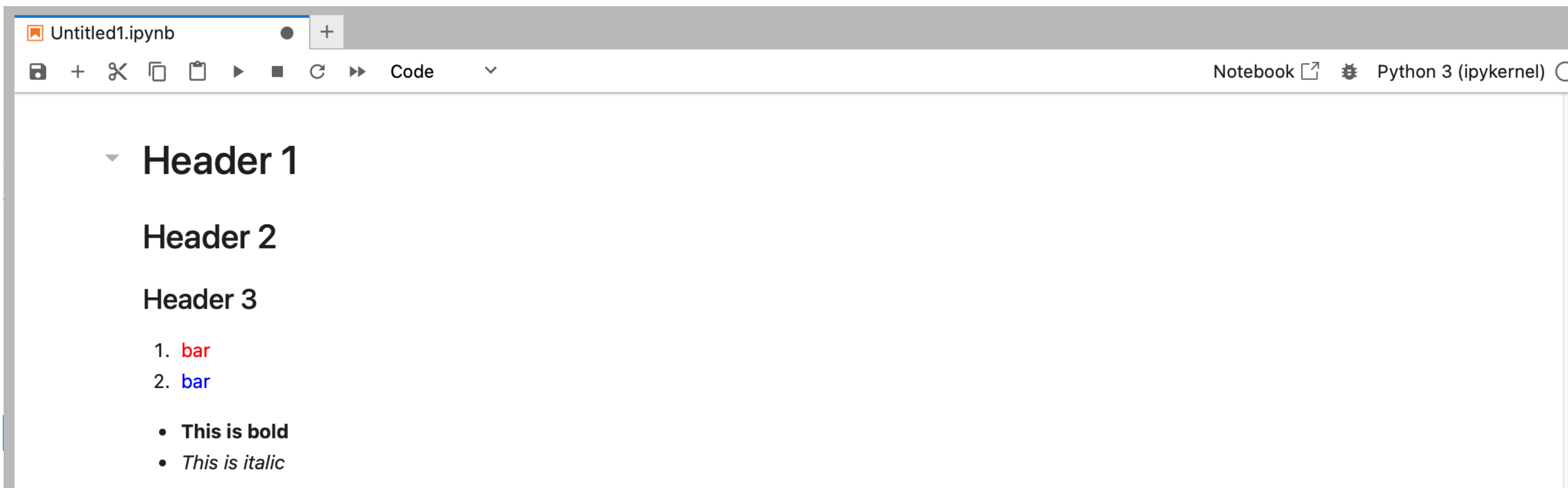


The image shows a Jupyter Lab window titled 'Untitled1.ipynb'. The top toolbar includes icons for saving, adding, deleting, copying, pasting, running, and a dropdown menu currently set to 'Markdown'. On the right, it says 'Notebook' and 'Python 3 (ipykernel)'. The main area contains a single cell with the following Markdown text:

```
# Header 1
## Header 2
### Header 3

1. <font color='red'>bar</font>
2. <font color='blue'>bar</font>

- __This is bold__
- _This is italic_
```



The image shows the same Jupyter Lab window, but the cell is now rendered. The top toolbar dropdown is now set to 'Code'. The rendered output of the Markdown cell is displayed below:

Header 1

Header 2

Header 3

- bar
- bar

- This is bold
- This is italic

Jupyter Lab: Multiple view option

Multiple view option using drag and drop

The screenshot displays the Jupyter Lab interface with a multi-view layout. On the left is the 'Launcher' sidebar containing a file browser with a search bar and a list of files and folders. The main area is divided into two panes. The left pane shows a Python code editor with the following content:

```
COL_NAMES = [  
    'age', 'class_of_worker', 'detailed_industry_recode', 'detailed_occupation_recode',  
    'education', 'wage_per_hour', 'enroll_in_edu_last_wk', 'marital_stat',  
    'major_industry_code', 'major_occupation_code', 'race', 'hispanic_origin',  
    'sex', 'member_of_labor_union', 'reason_for_unemployment', 'full_or_part_time_stat',  
    'capital_gains', 'capital_losses', 'dividends_from_stocks', 'tax_filer_stat',  
    'region_prev_residence', 'state_prev_residence', 'household_family_stat', 'household_summary',  
    'instance_weight', # Ignore  
    'msa_change_code', 'reg_change_code', 'move_within_reg', 'live_in_this_house_1_year_ago',  
    'prev_residence_sunbelt', 'num_persons_worked_for_employer', 'family_members_under_18',  
    'country_birth_father', 'country_birth_mother', 'country_birth_self', 'citizenship',  
    'own_business_or_self_employed', 'veterans_admin_questionnaire', 'veterans_benefits',  
    'weeks_worked_in_year', 'year', 'income'  
]  
  
MISSING_VALUES = [  
    '?',  
    'Not in universe',  
    'Not identifiable',  
    'NA',  
    'Do not know'  
]  
  
CAT_COLS = [  
    'detailed_industry_recode',  
    'detailed_occupation_recode',  
    'own_business_or_self_employed',  
    'veterans_benefits',  
    'year'  
]  
  
COLUMNS_TO_REMOVE = [  
    "reg_change_code",  
    "msa_change_code",  
    "move_within_reg",  
    "detailed_industry_recode",  
    "detailed_occupation_recode",  
    "household_family_stat"  
]
```

The right pane is a 'Notebook' view showing a preview of the code. A red dotted arrow indicates a drag-and-drop action from the file browser to the notebook pane. A tooltip for '02_Baseline experiments.ipynb' is visible over the notebook pane. The bottom status bar shows 'Simple' mode, 'Python 3 (ipykernel) | Idle', and the current file '02_Baseline experiments.ipynb'.

Jupyter Lab: cell move

The screenshot displays the Jupyter Lab interface with two notebooks open. The left sidebar shows a file explorer with a list of files: data, venv, 01_EDA..., 02_Base..., 03_Final..., and Untitled.... The main area shows two notebooks. The first notebook, 01_EDA.ipynb, contains a code cell with the following Python code:

```
[5]: cat_cols = train_df.select_dtypes(include=[object]).columns
# Print unique values for each categorical column
for col in cat_cols:
    print(f"\n◆ Column : {col}")
    uniques = train_df[col].unique()
    print(f"\nNb of unique values : {len(uniques)}")
    print(f"Values : {uniques}")
```

The second notebook, 02_Baseline experiments.ipynb, contains a code cell with the following Python code:

```
[2]: # Data configuration

COL_NAMES = [
    'age', 'class_of_worker', 'detailed_industry_recode',
    'education', 'wage_per_hour', 'enroll_in_edu_last_wk',
    'major_industry_code', 'major_occupation_code', 'race',
    'sex', 'member_of_labor_union', 'reason_for_unemployment',
    'capital_gains', 'capital_losses', 'dividends_from_stocks',
    'region_prev_residence', 'state_prev_residence', 'household_size',
    'instance_weight', # Ignore
    'msa_change_code', 'reg_change_code', 'move_within_region',
    'prev_residence_sunbelt', 'num_persons_worked_for_current_firm',
    'country_birth_father', 'country_birth_mother', 'country_birth_self',
    'own_business_or_self-employed', 'veterans_administration',
    'weeks_worked_in_year', 'year', 'income'
]

MISSING_VALUES = [
    '?',
    'Not in universe',
    'Not identifiable',
    'NA',
    'Do not know'
]

CAT_COLS = [
```

A red dashed box highlights the code cell in the second notebook, and a red arrow indicates its movement from the first notebook.

Easy cell move
using drag and
drop

Notebook file (.ipynb)

- Native Jupyter format
- Keeps code, text, and outputs
- Can be re-opened and edited in Jupyter

Python script (.py)

- Converts notebook into plain Python code
- Useful to transform experiments into reusable scripts

HTML (.html)

- Static web page with code, text, and results
- Easy to share (any browser can open it)

PDF (.pdf)

- Printable version of the notebook
- Often used for reports (requires LaTeX installed)



Google colab

- Cloud-hosted Jupyter Notebook environment by Google
- Runs directly in the browser, no installation needed
- **Key features**
 - Free access to **GPUs/TPUs** (limited quotas)
 - Easy sharing (similar to Google Docs)
 - Integrated with Google Drive for saving files
 - Pre-installed scientific libraries (NumPy, Pandas, TensorFlow, PyTorch, etc.)
- **Use cases**
 - Quick prototyping, collaborative notebooks, GPU training on small/medium datasets



Kaggle Notebooks

- Cloud-based Jupyter Notebooks hosted on Kaggle platform
- Runs directly in the browser, no installation needed
- **Key features**
 - Comes with free CPU/GPU/TPU compute (limited time per week)
 - Direct access to **Kaggle Datasets** and **Kaggle Competitions**
 - Built-in version control for notebooks
 - Rich community: share, fork, and reuse notebooks from others
 - No setup required, common ML/DL libraries pre-installed
- **Use cases**
 - Machine learning competitions
 - Data exploration with Kaggle datasets
 - Sharing reproducible experiments with the Kaggle community

Render jupyter notebook with voilà



Voilà turns Jupyter notebooks into standalone web applications.

<https://github.com/voila-dashboards/voila>

Installation

```
$ pip install voila
```

Add extension to jupyter

```
$ jupyter labextension install @voila-dashboards/jupyterlab-preview
```

Usage examples are available here:

<https://github.com/voila-dashboards/voila/tree/main/notebooks>